

AI Digital Workforce — Agent Development Specification

Version: 1.0 (Capstone Phase 2) Purpose: Development specification for the AI team building the specialized agents.

This document defines the architecture, responsibilities, inputs, outputs, and implementation contract for all AI agents. The backend team will integrate these agents through a FastAPI controller.

1. Project Overview

The system is a web-based AI Digital Workforce where a single Controller Agent communicates with multiple specialized AI agents. Each agent is responsible for one organizational role and returns structured JSON, not free-form markdown.

Architecture

2. Common Development Rules

Every agent must follow these rules

- Use OpenAI SDK or Claude SDK only.
- Do not use LangChain or LangGraph.
- Return valid JSON only.
- No Markdown.
- No HTML.
- No code fences.
- Every response must conform to its schema.

Common Python Interface

Code snippet -

```
class BaseAgent:
```

```
    async def run(
```

```
        query: str,
```

```
        context: dict
```

```
    ) -> dict:
```

```
        pass
```

Input

Field	Type	Description
query	string	User request
context	dict	Previous agent outputs or chat context

Output

Python dictionary matching the schema.

3. Business Analyst Agent

Objective

Convert a raw user idea into structured software requirements.

Responsibilities

- Identify the business problem
- Define target users
- Create functional requirements
- Create non-functional requirements
- Generate user stories
- List assumptions

Example Input

I want an AI chatbot for hospitals that helps patients book appointments.

Output Schema

JSON

```
{  
  "project_name": "",  
  "problem_statement": "",  
  "target_users": [],  
  "functional_requirements": [],
```

```
"non_functional_requirements": [],  
"user_stories": [  
  {  
    "role": "",  
    "goal": "",  
    "benefit": ""  
  }  
],  
"assumptions": []  
}
```

Prompt Guidelines

The agent must behave as a Senior Business Analyst.

It should:

- Never discuss implementation.
- Focus only on requirements.
- Generate maximum 10 functional requirements.
- Keep language concise and professional.

4. Solution Architect Agent

Objective

Transform business requirements into a high-level technical architecture.

Responsibilities

- Select system components
- Define frontend/backend responsibilities
- Identify databases
- List external services
- Produce Mermaid architecture

Input

Business Analyst JSON

Output Schema

JSON

```
{
  "architecture_summary": "",
  "frontend": "",
  "backend": "",
  "database": "",
  "external_services": [],
  "modules": [],
  "api_list": [],
  "mermaid": ""
}
```

Mermaid Requirement

The mermaid field must always contain a valid Mermaid flowchart.

Example:

```
graph TD
  A[React] --> B[FastAPI]
  B --> C[(Postgres)]
  B --> D[Redis]
  B --> E[OpenAI API]
```

Prompt Guidelines

Act as a Senior Solution Architect.

Focus on:

- Scalability
- Modularity
- Clear separation of components

Do not generate source code.

5. Software Engineer Agent

Objective

Generate the backend implementation blueprint.

Responsibilities

- Design REST APIs
- Suggest SQLAlchemy models
- Generate folder structure
- List environment variables
- List required dependencies

Input

Solution Architect JSON

Output Schema

JSON

```
{
  "folder_structure": [],
  "sqlalchemy_models": [],
  "api_endpoints": [
    {
      "method": "",
      "path": "",
      "purpose": ""
    }
  ],
  "environment_variables": [],
  "dependencies": []
}
```

Expected API Output

Example:

Method	Endpoint	Purpose
POST	/chat	Send user message
GET	/sessions	List chat sessions
GET	/messages/	Fetch conversation
POST	/projects	Save generated project

Prompt Guidelines -

Act as a Senior Backend Engineer.

Do not generate full source code.

Return implementation specifications only.

6. Project Manager Agent

Objective

Convert technical planning into an executable project roadmap.

Responsibilities

- Create milestones
- Break work into tasks
- Estimate duration
- Identify risks
- Define deliverables

Input

Outputs from all previous agents.

Output Schema

JSON

```
{  
  "epics": [],  
  "milestones": [],
```

```
"tasks": [  
  {  
    "title": "",  
    "priority": "",  
    "estimate_days": 0  
  }  
,  
  "risks": [],  
  "deliverables": []  
]
```

Example Task

JSON

```
{  
  "title": "Implement FastAPI authentication",  
  "priority": "High",  
  "estimate_days": 2  
}
```

Prompt Guidelines

Act as a Technical Project Manager.

Keep estimates realistic.

Maximum 20 tasks.

7. Controller Agent (Backend Team)

This agent is NOT part of the AI team.

It will be implemented inside FastAPI.

Responsibilities

- Detect user intent
- Select the correct specialist agent
- Pass context

TASK DISTRIBUTION WILL BE LIKE -

User Intent	Agent
Requirements	Business Analyst
Features	Business Analyst
Architecture	Solution Architect
Database	Solution Architect
API Design	Software Engineer
Backend	Software Engineer
Timeline	Project Manager

The controller contains deterministic Python logic and does not require LangGraph.

8. Folder Structure

The AI team should organize files as follows:

backend/

agents/

business_analyst.py

solution_architect.py

software_engineer.py

project_manager.py

prompts/

business.txt

architect.txt

engineer.txt

pm.txt

schemas/

analyst.py

architect.py

engineer.py

pm.py

services/

llm.py

9. SDK Recommendation

Use native SDKs only.

OpenAI

Python

Run

client.responses.parse(...)

Claude

Use structured JSON output with the Claude SDK.

No LangChain abstractions should be introduced.

10. Final Deliverables

Deliverable	Required
Agent Python file	✓
System prompt	✓
Pydantic JSON schema	✓
Sample input	✓
Sample JSON output	✓
README explaining the agent	✓